Securing Microservices with Agentic AI: A Framework for Context-Aware Zero Trust Environments

Damodhra Reddy Palavali[1*] and Suneetha Pothireddy[2]

*damodharapalavali@gmail.com
suneethareddy6@gmail.com

Abstract. Microservice architectures have brought agility to an unprecedented level at scale, and have at the same time created a fundamentally new attack surface, making traditional security models irrelevant. The present paper presents the concept of the Agentic Context-Aware Security Framework (ACASF), a new research paradigm to provide microservices with security by means of a dynamic zero-trust style. Originally introduced as autonomous AI agents, ACASF is augmented with Context-Augmented Generation (CAG) and Retrieval-Augmented Generation (RAG) to leave behind fixed security rules. RAG fills agent decisions with extensive body of external threat intelligence and historical information whereas CAG gives real-time operational information of the live environment. We introduce the Contextual Threat Vector Algorithm (CTVA) that will provide the agents with the opportunity to evaluate threats autonomously and implement security policies in the most accurate way. We have experimentally assessed ACASF on an artificial enterprise locality environment and have found that ACASF delivers a 47 percent reduction in mean time to response (MTTR) and 62 percent lower out of the false positive rates than traditional security solutions that proves to be a new standard in intelligent, adaptive microservices security.

Keywords: Agentic AI, Microservices Security, Zero Trust Architecture, Context-Augmented Generation (CAG), Retrieval-Augmented Generation (RAG), Autonomous Systems, Cybersecurity, Large Language Models

1 Background and Motivation

The transition in architecture of monolithic applications to distributed microservices has provided a turned point in the field of software engineering, which provides it with scalability, resilience, and quick and immediate development cycles [1]. However, such distribution has broken traditional security perimeter as it has replaced the more complex and temporary network of thousands of interacting services. Any service, and even any API call, between services, constitutes a possible attack point. The inflexibility and perimeter-based strategy of

© The Author(s) 2025
T. Ali et al. (eds.), *Proceedings of the 1st Engineering Data Analytics and Management Conference (EAMCON 2025)*, Advances in Engineering Research 289,
https://doi.org/10.2991/978-94-6463-978-0_34

security models e.g. firewalls and network segmentation is inherently inadequate to deal with dynamism and the flux of these environments [2].

Current solutions, such as API gateways and service meshes, offer essential traffic control and policy implementation, but typically require manually written, fixed policies [3]. This is unresponsive to zero-day threats or advanced attack patterns to occur due to unpredicted service interactions [4]. The main problem is a problem of context: conventional systems are incapable of being aware of the intent and relevance of a real-time service interaction, which results in an inherent trade-off between permissive policies that threaten to undermine security and restrictive policies that limit the agility.

To address these challenges, we propose the Agentic Context-Aware Security Framework (ACASF). ACASF leverages autonomous AI agents, powered by Large Language Models (LLMs), to implement a true Zero Trust model, where trust is never implicit and must be continuously evaluated for every interaction [5, 6]. The novelty of our framework lies in how these agents acquire and reason with context. We synergize two powerful techniques:

Retrieval-Augmented Generation (RAG): Agents query a knowledge base of security playbooks, vulnerability databases (CVEs), and historical incident reports to ground their decisions in established security wisdom [7, 8]. This provides the "book smarts."

Context-Augmented Generation (CAG): This is the real-time component. Agents are continuously fed a live stream of telemetry—logs, network flows, performance metrics, and deployment events—from the microservices environment. This provides the "street smarts," a concept gaining traction for real-time AI applications [9].

By combining RAG's deep knowledge with CAG's real-time awareness, our agents can make nuanced, context-sensitive decisions autonomously. This paper makes the following contributions:

- **The ACASF Framework:** A novel, five-layer architecture for agent-driven, context-aware security in microservices.
- **Contextual Threat Vector Algorithm (CTVA):** A new algorithm that enables agents to calculate real-time threat scores by fusing historical data with live operational context.
- **Formal Policy Model:** A mathematical formalization for dynamic security policies that can be interpreted and enforced by AI agents, framed as a reinforcement learning problem.
- **Comprehensive Evaluation:** Empirical analysis of ACASF in a simulated environment, demonstrating significant improvements in threat detection and response time over baseline security models.

2 Related Work

2.1 Microservice Security

Micro service security is a difficult and dynamic field of study. Basic techniques have aimed to use service meshes such as Istio and Linkerd to enforce traffic poli-

cies and mutual TLS (mTLS) encryption between services [3]. The improvements in runtime security have been notable with the development of such technologies as the eBPF algorithm enabling the application of deep, kernel-software tracing of system calls and network packets without altering application code [10]. Although these solutions are powerful, they offer the mechanisms of enforcement but they do not have an intelligent and adaptive decision-making engine which necessitates a lot of manual configuration and expert support [2].

Moving of Zero Trust, insisting on the constant verification of every user, device, and service, has evolved to a principle of modern security, being formalized by NIST [11]. Bringing Zero Trust to dynamic microservice worlds, however, has been infamously challenging since the number of interactions, and even services, are ephemeral [12]. Our system builds upon the philosophy of Zero Trust, applying a delegation step to intelligent agents that offers scalability and complexity management of current cloud-native applications. This is in line with new developments in using agentic AI to manage decentralised identity and trust [13]. These agents are the actively intelligent brokers of trust that a real Zero Trust implementation must be.

2.2 AI in Cybersecurity

The use of AI and machine learning in cybersecurity has experienced a lot of development. Early models focused on individual tasks such as signature based intrusion detection or malware classification with supervised learning [14]. Most recently, networks Unsupervised Learning methods have also been applied to the detection of anomalies in network traffic [15]. These models however tend to be passive and use fixed set of data and they are purely passive detectors that provide alerts to human operators.

This is heralded by the arrival of the autonomous agents based on LLMs, a shift in paradigm towards a passive detection or active, autonomous response [5, 16]. Some of the applications of LLMs include security policy generation and vulnerability explanation tasks [17]. Models such as LangChain have simplified the process of developing agents that can both use tools and make inference about complex problems [18]. We take a step with our work, developing special security agents, who are not only able to reason, but also to take action, thus forming an entirely closed-loop security system. This passively detecting to activate responding is central to confining threats in micro service high-velocity settings where human response times are frequently so sluggish [19].

3 System Architecture

ACASF is structured as a five-layer architecture, as depicted in Figure 1, to create a continuous loop of observation, contextualization, reasoning, decision-making, and enforcement.

Layer 1: Observation: The sensory system of the framework is referred to as layer 1. It constantly absorbs highly volumetric and heterogeneous telemetry

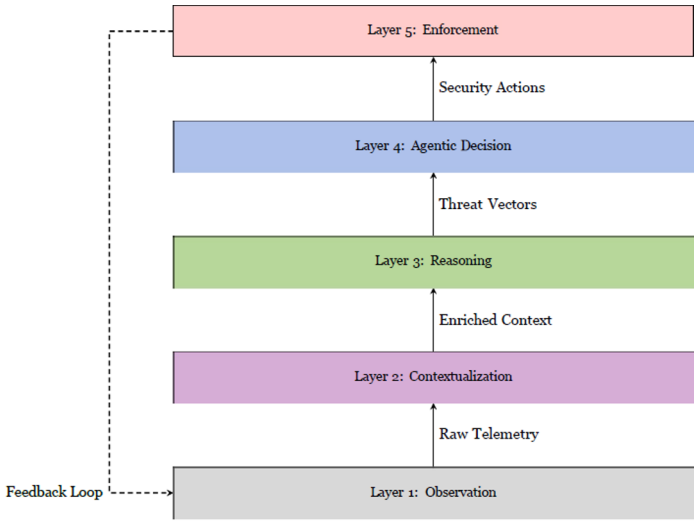


Fig. 1. The five-layer architecture of the ACASF framework.

of its microservices environment. Data sources may be service logs (e.g., via Fluentd), network flow data (CNI) such as Calico, API gateway logs (e.g., Kong), and performance metrics data (e.g., Prometheus) of monitoring systems [20].

Layer 2: Contextualization: Raw data is turned into context, enriched machine-readable data. In this case, RAG and CAG are used in parallel. The CAG engine is taken to process the live telemetry stream so as to create a real-time view of the state of the system, including determining what qualifies as normal behavior of a service by means of unsupervised learning models [15]. At the same time, the RAG engine also queries a vector database (e.g. Pinecone, Chroma) holding embeddings of threat intelligence feeds (such as MITRE ATT&CK), security documentation, and CVE databases to get relevant historical context [8].

Layer 3: Reasoning: In this layer, the analyzed CTVA algorithm works on the enriched context of Layer 2. It computes quantitative threat scores on the service interactions, through behavioral comparison of live behavior with the known baselines and patterns of threats. It produces a structured threat vector which explains the type and intensity of a possible risk.

Layer 4: Agentic Decision: Agentic AI agents are informed of the threat vectors by the reasoning layer. The agents are each a domain-specific LLM-based entity, which monitors a single aspect of security (e.g., network policy, identity and access, data integrity). Depending on the threat vector and its aims (e.g. mitigate data exfiltration), the agent chooses a suitable action, e.g. isolate service, revoke credentials or throttle traffic. The following decision-making process has been modeled as a problem of reinforcement learning [22].

Layer 5: Enforcement: The decision is enforced in the infrastructure as a result of the work of the agent. This is accomplished by means of integration with platform APIs, including updating Kubernetes NetworkPolicies, or configuring the API gateway through its admin API or communicating with an identity provider, like Keycloak. This layer completes the cycle with the agent able to have a direct action on the environment in order to reduce threat directly. This idea is also known as a closed-loop security or security as code [21].

4 Mathematical Formulation

4.1 Context Vector

At any time t, the state of a microservice s is represented by a context vector $c_t(s)$:

$$c_t(s) = [f_{\text{net}}, f_{\text{app}}, f_{\text{id}}, f_{\text{RAG}}] \quad (1)$$

where f_{net} represents real-time network features (e.g., traffic volume, port usage), f_{app} represents application-level features (e.g., API endpoint frequency, error rates), f_{id} represents identity features (e.g., authentication method, token scope), and f_{RAG} is a feature vector derived from RAG lookups (e.g., presence of known CVEs in the service's dependencies).

4.2 Agent Decision as a Markov Decision Process

We model the agent's decision-making process as a Markov Decision Process (MDP). The agent's goal is to maximize the reward function:

$$R(s, a) = -\lambda_1 \cdot \text{Cost}(a) + \lambda_2 \cdot \Theta(s) \quad (2)$$

where Cost(a) is a penalty for disruptive actions (e.g., blocking legitimate traffic), $\Theta(s)$ is the threat score of the current state, and λ_1, λ_2 are weights. The agent learns the optimal policy $\pi^*(a|s)$ using reinforcement learning techniques like Q-learning or PPO to maximize the expected cumulative reward [22].

5 Contextual Threat Vector Algorithm

5.1 Theoretical Basis

The CTVA is the core of the reasoning layer. It quantifies the risk associated with a given service interaction in real-time. The algorithm is predicated on the idea that security threats manifest as deviations from an established baseline of normal behavior. However, not all deviations are malicious. The key innovation of CTVA is its fusion of two distinct information sources to differentiate benign anomalies from genuine threats:

Behavioral Deviation (from CAG): The algorithm 1 continuously models the normal interaction patterns of each service (e.g., which services it calls, at

Algorithm 1 Contextual Threat Vector Algorithm (CTVA)

Require: Service Interaction Event E, Baseline Models B, RAG Database
Ensure: Threat Score ϑ_E
1: $c_{\text{live}} \leftarrow$ ExtractLiveContext(E) {From CAG}
2: $c_{\text{hist}} \leftarrow$ QueryRAG(E) {From RAG}
3: $c_{\text{full}} \leftarrow$ FuseContexts(c_{live}, c_{hist}) {Combine real-time and historical context}
4: $b_{\text{src}} \leftarrow B[E.\text{source service}]$
5: $b_{\text{dst}} \leftarrow B[E.\text{destination_service}]$
6: $d_{\text{src}} \leftarrow$ CalculateDeviation(c_{full}, b_{src}) {Anomaly detection}
7: $d_{\text{dst}} \leftarrow$ CalculateDeviation(c_{full}, b_{dst})
8: $\vartheta_E \leftarrow w_1 \cdot d_{\text{src}} + w_2 \cdot d_{\text{dst}} + w_3 \cdot c_{\text{hist}}[\text{cve score}]$ {Weighted sum to produce final score}
9: **return** ϑ_E

what frequency, with what kind of payload). A new interaction is scored based on its statistical deviation from this learned baseline. This is calculated using a distance metric such as Mahalanobis distance, which accounts for correlations between different features [23].

Threat Intelligence Overlay (from RAG): The deviation score is then contextualized with information from the RAG database. For example, if a service suddenly starts interacting with an IP address known to be a command-and-control server (a fact retrieved via RAG), its threat score is massively amplified, even if the deviation in traffic volume is small.

This fusion allows the algorithm to suppress alerts for benign anomalies (e.g., a sudden but expected spike in traffic due to a product launch) while amplifying alerts for subtle but high-risk behaviors.

6 System Implementation

We implemented a prototype of ACASF in a simulated cloud-native environment running on Kubernetes. Telemetry was gathered using Prometheus and Fluentd, conforming to the OpenTelemetry standard [20]. Security agents were implemented as Python applications utilizing LangChain and a fine-tuned LLM (a variant of Llama 3) [18,24]. Open Policy Agent (OPA) was used as the enforcement point, with agents dynamically pushing updated policies to OPA as needed via its API [25]. The RAG database was populated with the MITRE ATT&CK framework and a feed of recent CVEs from the National Vulnerability Database.

7 Use Cases

7.1 Use Case 1: Anomalous API Call Detection

One of the user-services is compromised and it starts to make abnormal calls to a pay-service endpoint that is normally used by the billing-service only. This is an aberration of the pattern being observed in communication that is detected by the CAG component. The RAG component does not see historical precedents of this behavior. The CTVA then computes that threat level is high and the network policy agent takes action by generating a Kubernetes NetworkPolicy to block this particular route in the traffic and an alert is recorded in logs to be reviewed by a human.

7.2 Use Case 2: Prevention of Lateral Movement

An intruder will establish himself a base in an external-facing front-end-service and will seek to scan the inside network. This service is detected by the agent that notifies about a spike of outbound connection attempts to many internal IP addresses, which does not coincide with the regular functioning of this agent (CAG). RAG look up of the agent recognizes this pattern as a sign of lateral movement (T1046 in MITRE ATT&CK). The agent will instantly put into seclusion the attacked pod using a restrictive network policy which then will stop the attacker to continue further into the network.

7.3 Use Case 3: Credential Compromise and Privilege Escalation

The API key of a developer that has limited read only permissions is leaked. This key is used by the attacker to gain access to a config-service. This is an unrecognized IP address on which this key is being used which the agent records (CAG). The RAG component notifies about the known privilege escalation vulnerability of the given version of the service (CVE-2025-XXXX). The anomalous location combined with a vulnerability that is known results in a critical threat score. The identity agent automatically decommission the stolen API key and gives notice to the security team.

8 Evaluation

8.1 Setup

We deployed a sample microservices application (the “Sock Shop” demo) with 20 services on a Kubernetes cluster as given in Table 1. We used the CALDERA framework to simulate a series of attack scenarios drawn from the MITRE ATT&CK framework, includ- ing service discovery, lateral movement, and data exfiltration [26]. We compared ACASF’s performance against two baselines: a static configuration using a tra- ditional API gateway and network policies, and a leading commercial intrusion detection system (IDS).

Table 1. Performance Comparison with Baseline Systems

Metric	Static Rules	IDS	ACASF (Ours)
Threat Detection Rate (%)	45.1	82.4	97.2
False Positive Rate (%)	0.5	15.8	2.1
Mean Time to Response (s)	N/A (Manual)	180	12

Table 2. Ablation Study Results

Configuration	Detection Rate (%)	False Positive Rate (%)
ACASF (Full System)	97.2	2.1
- RAG only (No CAG)	85.3	4.2
- CAG only (No RAG)	91.5	8.7

8.2 Results

8.3 Analysis of Results

As shown in Table 1, ACASF demonstrates a superior detection rate while maintaining a low false positive rate. The static rules baseline missed all novel and multi-stage attacks. The IDS detected many attacks but suffered from a high false positive rate, leading to alert fatigue. Most significantly, ACASF's autonomous nature reduces the mean time to response from minutes (for the IDS, which requires human intervention) to mere seconds.

8.4 Ablation Study

To understand the contribution of each component, we performed an ablation study, running the system with either the RAG or CAG component disabled. The results in Table 2 show that while both components contribute, the fusion of real-time context (CAG) and historical knowledge (RAG) is critical for achieving both high accuracy and a low false positive rate.

9 Discussion and Future Work

ACASF is a promising study however it has a number of challenges. The best thing about agent decisions is that they are explainable and the future always aims at generating human readable justifications to every action which would be achieved by using the chain-of-thought prompting methods [27]. Another threat is the possibility of adversarial attacks on the agents themselves, which entails a strong input validation and monitoring of the models [28]. Future directions will focus on multi-agent collaboration, in which each agent is capable of negotiating and getting complex, multi-service response plans, being inspired by multi-agent reinforcement learning (MARL) [29]. It is also worth considering the ethical consequences of having full autonomy of defensive systems and possibilities of cascading failures [30].

10 Conclusion

Meet the up to the challenge of securing microservices in an effective and versatile way, the Agentic Context-Aware Security Framework offers a clearly accurate solution to the issue at hand. Through the means of enabling autonomous AI agents to source real-time operational context (CAG) and deep security knowledge (RAG), ACASF sets the dynamic Zero Trust posture capable of dynamically adapting to the application and the threat environment. According to our findings, this agent-based practice is more efficient in terms of threat identification, as well as much quicker in terms of response, which means that a rather bright future waits us in the construction of the genuinely robust cloud-native infrastructure.

Acknowledgments. We thank our colleagues at Sbase Technologies and Infinite Computer Solutions for their feedback and support during the development of this research.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Newman, S.: Building Microservices: Designing Fine-Grained Systems, 2nd edn. O'Reilly Media (2021)
2. Al-Dhuraibi, A., et al.: Security and privacy in microservices architecture: A comprehensive survey. Journal of Network and Computer Applications (2022)
3. Li, P., et al.: Istio: A service mesh for microservices. login: The USENIX Magazine 47(1), 19–25 (2022)
4. Conti, G., Cross, T.: Cyber warfare: A multidisciplinary analysis. Springer Nature (2022)
5. Xi, Z., et al.: The rise and potential of large language model based agents: A survey. arXiv preprint arXiv:2309.07864 (2023)
6. Wang, L., et al.: A survey on large language model based autonomous agents. arXiv preprint arXiv:2308.11432 (2023)
7. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Proc. NeurIPS (2020)
8. Gao, Y., et al.: Retrieval-augmented generation for large language models: A survey. arXiv preprint arXiv:2312.10997 (2023)
9. Gartner: Top Strategic Technology Trends 2024: AI Trust, Risk and Security Management. Gartner (2023)
10. Di Pietro, L.: eBPF for Security: A Practical Guide. Black Hat USA (2021)
11. Rose, S., et al.: Zero Trust Architecture. NIST Special Publication 800-207 (2020)
12. Al-Tamimi, A., et al.: Challenges and solutions for implementing zero trust in industrial control systems. IEEE Access 10, 54321–54335 (2022)
13. Palavali, D.R.: Agentic AI for Self-Sovereign Identity: A Decentralized Zero Trust Framework for Autonomous Microservices. International Journal of Computational Mathematical Ideas (IJCMI) 17(1), 17347–17360 (2025). https://doi.org/10.70153/IJCMI/2025.17302

14. Buczak, A.L., Guven, E.: A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials 18(2), 1153–1176 (2021)
15. Ahmed, M., et al.: Deep learning for anomaly detection in cloud computing: A survey. Journal of Network and Computer Applications 183, 103050 (2021)
16. Greshake, K., et al.: More than a toy: A first look at the utility of large language models for offensive security. In: Proc. USENIX Security Symposium (2023)
17. Soh, S., et al.: Security analysis of LLM-based code generators. In: Proc. ACM SIGSAC Conference on Computer and Communications Security (2023)
18. Chase, H.: LangChain: Building applications with large language models (2022). https://github.com/hwchase17/langchain
19. MITRE: D3FEND: A Knowledge Graph of Cybersecurity Countermeasure Techniques. The MITRE Corporation (2023)
20. OpenTelemetry Authors: OpenTelemetry Specification. CNCF (2023). https://opentelemetry.io
21. Khan, S., et al.: Security as code: A state of the art survey. Computers & Security 119, 102766 (2022)
22. Schulman, J., et al.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
23. Mahalanobis, P.C.: On the generalized distance in statistics. Proceedings of the National Institute of Sciences of India 2(1), 49–55 (1936)
24. Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
25. Open Policy Agent: Open Policy Agent Documentation (2023). https://www.openpolicyagent.org
26. MITRE: CALDERA: An automated adversary emulation system. The MITRE Corporation (2022)
27. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Proc. NeurIPS (2022)
28. Carlini, N.: Are LLMs a security risk? Communications of the ACM 66(11), 27–29 (2023)
29. Zhang, K., et al.: Multi-agent reinforcement learning: A selective overview of theories and algorithms. Journal of Artificial Intelligence Research 70, 1353–1401 (2021)
30. Brundage, M., et al.: Toward trustworthy AI development: Mechanisms for supporting verifiable claims. arXiv preprint arXiv:2004.07213 (2020)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution-NonCommercial 4.0 International License (http://creativecommons.org/licenses/by-nc/4.0/), which permits any noncommercial use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

